

Theoretical Definition:

In Python, **magic methods** (also known as **special methods** or **dunder methods**) are functions with double underscores at the beginning and end (e.g., `__method__`). They allow classes to **behave like built-in types** and interact naturally with Python's syntax and operators.

Common Magic Methods:

Magic Method

Purpose

`__init__`

Initializes an object (constructor)

`__str__`String representation for `print()``__repr__`

Official string representation (for debugging)

`__len__`Used by `len()``__getitem__`Indexing support like `obj[i]``__add__`Defines behavior for the `+` operator`__eq__`, `__lt__`Comparison operations (`==`, `<`, etc.)`__iter__`, `__next__`

Custom iterator support

Practical Example:

python

 Copiar

 Editar

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Point({self.x}, {self.y})"

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

# Create two points
p1 = Point(2, 3)
p2 = Point(4, 1)

print(p1)           # Calls __str__: Point(2, 3)
print(p1 + p2)      # Calls __add__: Point(6, 4)
print(p1 == p2)     # Calls __eq__: False
```

🧠 Why Are Magic Methods Useful?

- They make your custom classes **behave like native Python objects**.
- Improve **readability and expressiveness** of your code.
- Essential for **advanced patterns** like custom iterators, context managers (`with`), sorting, operator overloading, etc.